

Simulació Probabilística del Joc de l'Oca

Pràctica 7

Serafí Nebot Ginard
Universitat de les Illes Balears (UIB)
Grau en Enginyeria Informàtica
Algorismes Avançats, Curs 2025–2026

Resum—Aquesta memòria descriu la implementació d'una simulació Monte Carlo per estimar el nombre de torns necessaris per acabar una partida amb un únic jugador del Joc de l'Oca. El programa implementa les regles exactes de l'enunciat, incloent-hi el rebot a la casella final, les caselles especials, les penalitzacions i la regla que limita cada tirada a un únic efecte especial. La solució s'ha organitzat amb arquitectura Model-Vista-Controlador, una entrada de consola amb el format de sortida requerit i una interfície gràfica Swing que permet executar simulacions, visualitzar estadístics i mostrar un histograma empíric. També es documenta el criteri de percentils emprat, el cost computacional i una validació experimental amb llavors fixes.

Termes d'índex—Monte Carlo, algorismes probabilístics, simulació, percentils empírics, MVC, Java Swing, Joc de l'Oca.

1. Introducció

L'objectiu de la pràctica és estimar mitjançant simulació el nombre de torns que necessita una partida del Joc de l'Oca per arribar a la casella final. El problema es podria modelar de manera exacta com una cadena de Markov amb estats que incloguessin la posició i les penalitzacions pendents. Tot i això, l'enunciat demana explícitament una aproximació per simulació Monte Carlo, que és més simple d'implementar i permet obtenir estimacions empíriques a partir d'un nombre gran de partides independents.

La implementació resultant té dos modes d'ús. La classe `Main` compleix el contracte de consola: llegeix un enter N , simula N partides i escriu els estadístics amb les etiquetes indicades. A més, la classe `GuiMain` obre una interfície Swing que reutilitza el mateix model de simulació i afegeix visualització tabular i gràfica dels resultats.

2. Fonaments Teòrics

2.1. Estimació Monte Carlo

Sigui X la variable aleatòria que representa el nombre de torns comptats en una partida completa. Una simulació de N partides genera una mostra independent $x_1, x_2, \dots, x_N$. La mitjana mostrada

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i \quad (1)$$

és l'estimador Monte Carlo de l'esperança $E[X]$. Quan N creix, la llei dels grans nombres justifica que aquesta mitjana s'aproximi al valor esperat real [1].

2.2. Percentils empírics

Per als percentils s'ha emprat el criteri de rang més proper. Si la mostra ordenada és $y_1 \leq y_2 \leq \dots \leq y_N$, el percentil empíric p es defineix com

$$Q_p = y_{\lceil pN \rceil}. \quad (2)$$

Aquest criteri retorna sempre un valor observat de la mostra, fet coherent amb la sortida sencera esperada per als mínims, màxims, percentils i mediana. La mitjana és l'únic valor que es mostra amb sis decimals.

2.3. Cost computacional

Si T és el nombre mitjà de torns d'una partida, simular N partides té cost esperat $O(NT)$. Després cal ordenar els N valors observats per calcular els percentils, amb cost $O(N \log N)$. Per tant, el cost total és

$$O(NT + N \log N). \quad (3)$$

La memòria emprada és $O(N)$, ja que es conserva la mostra sencera per poder ordenar-la i dibuixar l'histograma de la interfície gràfica.

3. Disseny i Implementació

3.1. Model de regles

El tauler s'ha modelat amb caselles enteres de 0 a 63. En cada tirada normal es calcula primer la posició provisional $p + d$. Si aquesta posició supera la casella final, s'aplica el rebot especificat a l'enunciat:

$$q = 63 - ((p + d) - 63). \quad (4)$$

Després del moviment i del possible rebot s'aplica, com a màxim, un únic efecte especial. La taula 1 resumeix els efectes implementats.

Taula 1. CASELLES ESPECIALS IMPLEMENTADES.

Tipus	Efecte
Oca	Avança a la següent oca; la nova tirada no suma torn
Pont	$6 \leftrightarrow 12$
Dados	$26 \leftrightarrow 53$
Posada	1 torn anul·lat
Pou	2 torns anul·lats
Laberint	$42 \rightarrow 30$
Presó	3 torns anul·lats
Mort	$58 \rightarrow 0$

La regla d'un sol efecte per tirada és important. Per exemple, si una tirada porta el jugador a la casella 6, s'aplica el pont fins a la casella 12, però no s'aplica immediatament l'efecte de la casella 12. L'única excepció és arribar a la casella 63, cas en què la partida acaba immediatament.

3.2. Arquitectura MVC

La separació Model-Vista-Controlador s'ha mantingut explícita. El model no importa cap classe de Swing i concentra les regles i els càlculs estadístics. La vista no conté lògica de joc, sinó que emet esdeveniments i renderitza els resultats que rep. El controlador és l'únic component que coneix simultàniament el model i la vista.

Taula 2. RESPONSABILITATS PRINCIPALS DE LES CLASSES.

Classe	Responsabilitat
OcaRules	Moviment, rebot i efectes especials
OcaGameSimulator	Simulació d'una partida completa
MonteCarloSimulation	Execució de N partides independents
SimulationStats	Mínims, percentils, mediana, màxim i mitjana
SimulationController	Llançament de simulacions des de la GUI
SimulationFrame	Formulari, taula de resultats i sortida requerida
HistogramPanel	Visualització de la distribució empírica

Les simulacions de la interfície gràfica s'executen dins un `SwingWorker`. Això evita bloquejar el fil d'esdeveniments de Swing mentre es calculen moltes partides. El mateix model es fa servir des de consola i des de la GUI, de manera que el format obligatori de sortida i la representació gràfica provenen de les mateixes dades.

3.3. Simulació d'un torn

La classe `OcaGameSimulator` incrementa el comptador una vegada per cada torn comptat. Si hi ha penalitzacions pendents, el torn es consumeix sense llançar el dau. En cas contrari es tira el dau i, si l'efecte d'una oca dona dret a una nova tirada, aquesta nova tirada es processa dins el mateix torn comptat.

```

1 int turns = 0;
2 GameSnapshot state = new GameSnapshot(0, 0);
3 while (!state.isFinished()) {
4     state = advanceTurn(state);
5     turns++;
6 }

```

Listing 1. Estructura bàsica de la simulació d'una partida.

La injecció de la interfície `Die` permet utilitzar un dau aleatori en producció i seqüències deterministes en les proves unitàries. Aquesta decisió fa que les regles especials es puguin verificar sense dependre de l'atzar.

4. Resultats i Discussió

4.1. Validació funcional

S'han implementat proves unitàries amb JUnit 5 per als casos més sensibles de l'enunciat: rebot després de la casella 63, finalització exacta, oques amb tirada extra, ponts i dados sense cascada d'efectes, penalitzacions que consumeixen torns sense llançar el dau, laberint, mort i càlcul dels percentils. La suite actual conté 13 proves i s'ha executat correctament amb `mvn test`.

També s'ha validat manualment el mode de consola. Per exemple, amb entrada N , la sortida es construeix sempre amb les etiquetes exactes demanades:

```

1 Mínimo: ...
2 P1: ...
3 P2: ...
4 P3: ...
5 P4: ...
6 Mediana: ...
7 P6: ...
8 P7: ...
9 P8: ...
10 P9: ...
11 Máximo: ...
12 Media: ...

```

Listing 2. Format de sortida generat per `SimulationStats`.

4.2. Experiments

La taula 3 mostra tres execucions amb llavors fixes. Per a cada N s'ha emprat la llavor $20260609 + N$, de manera que els valors són reproduïbles amb el mateix generador pseudoaleatori de Java. Els temps són mesures orientatives obtingudes en l'entorn local de desenvolupament.

Els resultats indiquen que la mitjana s'estabilitza al voltant de 20,7 torns a mesura que augmenta N . Els percentils centrals també són molt estables: en les tres execucions el percentil 0,1 és 8, la mediana és 17 i el percentil 0,9 és 38. En canvi, el màxim observat creix quan es fan més partides. Aquest comportament és esperable, perquè el màxim és molt sensible a la cua de la distribució i no és un estimador estable del comportament típic.

Taula 3. RESULTATS EMPÍRICS DE LA SIMULACIÓ MONTE CARLO.

N	Mínim	$P_{0.1}$	Mediana	$P_{0.9}$	Mitjana	Màxim	Temps (ms)
1.000	3	8	17	38	20.425000	107	8
10.000	3	8	17	38	20.651200	143	11
100.000	2	8	17	38	20.704830	175	116

5. Conclusions

La pràctica s'ha resolt amb una simulació Monte Carlo directa i modular. El model implementa les regles exactes de la partida, incloent les penalitzacions i la regla d'un sol efecte especial per tirada. La separació MVC permet disposar d'una sortida de consola estricta i d'una interfície gràfica sense duplicar la lògica de simulació.

Els experiments mostren que amb $N = 100.000$ partides la mitjana i els percentils principals ja són estables per a una estimació pràctica. Com a ampliacions naturals es podrien afegir simulació concurrent, comparació de variants de regles, recompte de freqüències de visita de cada casella o estudi de partides amb diversos jugadors.

Referències

- [1] C. P. Robert and G. Casella, *Monte Carlo Statistical Methods*. Springer, 2004.
- [2] Oracle, “Worker Threads and SwingWorker,” *The Java Tutorials*. [En línia]. Disponible: <https://docs.oracle.com/javase/tutorial/uiswing/concurrency/worker.html>